

Git Flow Classic

Цель

Этот документ описывает правила работы с Git-ветками в команде.

Документ нужен для того, чтобы:

- зафиксировать единый подход к созданию и merge веток;
- отделить production-код от текущей разработки;
- сделать процесс подготовки релиза более управляемым;
- формализовать работу с ветками;
- сделать состав релиза и production-состояние кода более прозрачными.

Структура



Branches

- **main** - стабильная production-версия;
- **develop** - основная ветка разработки;
- **feature/*** - ветки для новых задач;
- **release/*** - ветки подготовки и стабилизации релиза;
- **hotfix/*** - ветки срочных исправлений production;
- **bugfix/*** - ветки исправления ошибок в рамках текущей разработки или релизной ветке.

main

Стабильная production-ветка.

В `main` должен находиться только код, который:

- уже выпущен в production;
- либо полностью готов к выпуску в production.

Правила:

- прямые коммиты запрещены;
- изменения попадают только через merge из `release/*` или `hotfix/*`;
- каждый production-релиз(мердж в майн) должен быть отмечен тегом (v1.0.0).

`develop`

Основная ветка разработки. В эту ветку попадают завершённые задачи, которые должны войти в следующий релиз.

Правила:

- прямые коммиты запрещены;
- изменения попадают через Merge Request;
- от этой ветки создаются `feature/*`, `bugfix/*` и `release/*`;
- ветка должна быть работоспособной и проходить тесты.

`feature/*`

Ветки для разработки новых функциональностей. Создаются от `develop` и мержаются обратно в `develop`

Правила:

- одна ветка - одна задача;
- после завершения создаётся Merge Request в `develop`;
- Перед мерджем обязательно пройти Code Review.
- после merge ветка удаляется.

Перед созданием Merge Request автор должен:

- обновить ветку от `develop`;
- проверить, что проект собирается;
- проверить happy path;
- добавить или обновить тесты, если изменение затрагивает бизнес-логику;
- заполнить описание MR;
- приложить ссылку на задачу.

`release/*`

Ветки подготовки релиза. Создаются от `develop` мержаются в `develop` и `main`

Назначение:

- финальное тестирование;
- исправление найденных дефектов;
- обновление версии приложения;
- стабилизация релиза.

Правила:

- новая функциональность в `release/*` не добавляется;
- допускаются только `bugfix`, документация, конфигурация и подготовка релиза;
- при `merge` в `main` создаётся `tag`.

В `release/*` запрещены:

- новые `feature`-задачи;
- крупный рефакторинг;
- изменения, не связанные с подготовкой конкретного релиза;
- экспериментальные изменения.

`bugfix/*`

Ветки исправления ошибок в рамках текущей разработки или в рамках подготовки релиза.

Создаются от:

- `develop` - если ошибка найдена в рамках текущей разработки;
- `release/*` - если ошибка найдена во время стабилизации конкретного релиза.

Мержатся обратно в ту ветку, от которой были созданы.

В `bugfix/*` разрешены:

- исправление дефекта;
- тесты на исправленный сценарий;
- минимальные изменения, необходимые для исправления.

В `bugfix/*` запрещены:

- новая функциональность;
- крупный рефакторинг;
- изменения, не связанные с дефектом.

`hotfix/*`

Ветки срочных исправлений `production`. Создаются от `main` мержаются в `develop` и `main`

Правила:

- используется только для `production`-инцидентов;

- после merge в main создаётся новый tag;
- исправление обязательно должно попасть обратно в develop.
- Если есть активная `release/*`, в которую тоже нужно внести исправление, hotfix должен быть перенесён и туда.

Branch naming convention

Для названия веток используется единый формат:

```
<type>/<task-or-version>
```

Type	Example	Назначение
feature	feature/SRHR-1234	новая функциональность
bugfix	bugfix/SRHR-1235	исправление ошибки
release	release/v1.1.0	подготовка релиза
hotfix	hotfix/v1.1.1	срочное исправление production

Ответственность

Автор изменения отвечает за:

- создание ветки от правильной исходной ветки;
- актуальность своей ветки;
- корректную целевую ветку MR;
- описание MR;
- обработку замечаний;
- проверку happy path.

Reviewer отвечает за:

- проверку корректности решения;
- проверку рисков;
- проверку соответствия Git Flow;
- своевременную обратную связь.

Ответственный за релиз отвечает за:

- создание `release/*`;
- контроль состава релиза;
- `merge release/* в main`;
- создание tag;
- возврат изменений в `develop`;
- уведомление команды о выпуске.